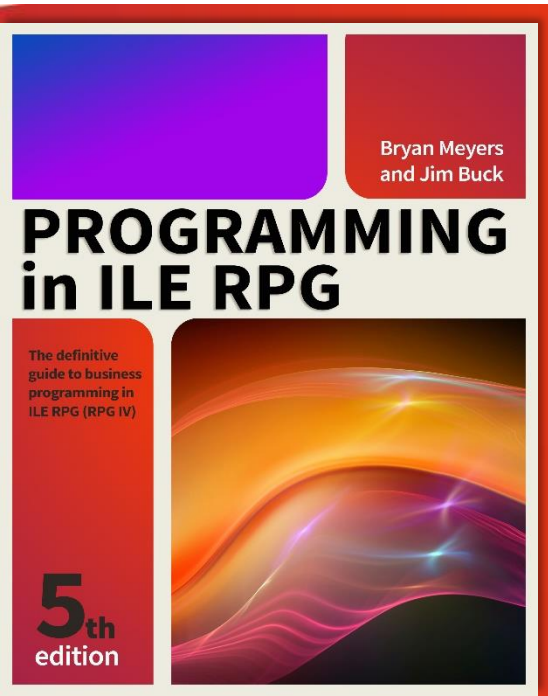
Chapter 7:Processing Character Data



Presentation © Copyright 2015, Enskill.com

Simple Character Assignment

- Eval (Evaluate Expression) operation assigns value to variable
  - Target variable and expression must be character data
  - Starts from left

```
Email = 'jdoe@mydomain.com';
Disclaimer = 'This report contains proprietary and confidential +
                          information, and is for internal use only.‘
Prefix = Coursename;
```

When working with long literals such as in this example, a “continuation” character is used:
•A hyphen (-) indicates continuation is in the first available position in the continued field
•A plus (+) indicates continuation with the first non-blank character in or past the first position in the continued field

Simple Character Assignment

- If target variable is longer than assigned value, Eval pads variable with blanks
  - If target is too small, Eval truncates extra rightmost characters

```
Dcl-s Var1 Char(7);
Dcl-s Var2 Char(12);
Dcl-s Var3 Char(12) Inz('mnopqrstuvwx');
Dcl-s Var4 Char(4);

Var1 = 'ABCDEFG';      // Var1 = 'ABCDEFG'
Var2 = 'ABCDEFG';      // Var2 = 'ABCDEFG     '
Var3 = 'ABCDEFG';      // Var3 = 'ABCDEFG     '
Var4 = 'ABCDEFG';      // Var4 = 'ABCD'
```

Evalr (Evaluate Expression, Right adjust)

- Evalr works like Eval for character expressions, except it right adjusts result
  - Starts with rightmost character

```
Dcl-s Var1 Char(7);
Dcl-s Var2 Char(12);
Dcl-s Var3 Char(12) Inz('mnopqrstuvwx');
Dcl-s Var4 Char(4);

Evalr Var1 = 'ABCDEFG';     // Var1 = 'ABCDEFG'
Evalr Var2 = 'ABCDEFG';     // Var2 = '     ABCDEFG'
Evalr Var3 = 'ABCDEFG';     // Var3 = '     ABCDEFG'
Evalr Var4 = 'ABCDEFG';     // Var4 = 'DEFG'
```

Eval-corr (Assign corresponding subfields)

- Eval-corr (Assign Corresponding Subfields) operation assigns subfields of one data structure to corresponding subfields of another data structure
  - Assigns subfields of one data structure to corresponding subfields of another
  - Subfield names must be same
  - Subfield data attributes must be compatible
  - At least one data structure must be qualified
  - Eval-Corr ignores subfields it cannot assign

Eval-corr (Assign corresponding subfields)

```
Dcl-ds Customer Qualified;
  Identifier  Zoned(5:0);
  Name        Char(30);
  State       Char(2);
  Zipcode     Char(10);
  Phone       Char(10);
  Creditlimit Zoned(11:2);
End-ds;

Dcl-ds Vendor Qualified;
  Identifier Uns(5);
  Name       Char(35);
  Address    Char(35);
  City       Char(21);
  State      Char(2);
  Postcode   Char(10);
  Phone      Zoned(10:0);
End-ds;

Eval-corr Customer = Vendor;
```

Vendor (Source)	***Customer (Target)***	***Action***
Identifier	Identifier	Subfield assigned; both subfields are numeric, even though they are different data types
Name	Name	Subfield assigned, with truncation
Address	--	No target subfield; subfield ignored
City	--	No target subfield; subfield ignored
State	State	Subfield assigned
Postcode	Zipcode	Different subfield names; subfields ignored
Phone	Phone	Incompatible data types; subfields ignored
--	Creditlimit	No source subfield; subfield ignored

```
// Same as:
     Customer.Identifier = Vendor.Identifier;
     Customer.Name = Vendor.Name;
     Customer.State = Vendor.State;
```

Assigning Values with Figurative Constants

- *Blank (or *Blanks) fills variable with blanks
- *Zero (or *Zeros) fills variables with zeros
- *Hival is all bits on (X'FF')
- *Loval is all bits off (X'00')

```
Dcl-s Var       Char(9);

Var = *Blanks;          // Var = '         '
Var = *Zeros;           // Var = '000000000'
Var = *Hival;           // Var is now filled with X'FF' characters
Var = *Loval;           // Var is now filled with X'00' characters
```

Assigning Values with Figurative Constants

- *All repeats string through entire length of result
  - *Allx specifies hexadecimal character strings

```
Dcl-s Var       Char(9);
Dcl-s Underline Char(80);

Var = *All'Z';          // Var = 'ZZZZZZZZZ'
Var = *All'Abc';        // Var = 'AbcAbcAbc'
Var = *Allx'E9'         // Var = 'ZZZZZZZZZ' (X'E9' = Z)
Var = *Allx'C18283'     // Var = 'AbcAbcAbc' (X'C18283' = Abc)
Underline = *All'-'     // Underline is now 80 hyphens
```

Concatenating Character Values

- + operator concatenates (joins) two or more character values
  - Operands can be variables, literals, or constants
  - Must be character data
  - Works with Eval and Evalr

```
Dcl-s Greeting   Char(12);
Dcl-s Fullname   Char(16);
Dcl-s Firstname  Char(5)  Inz('Susan');
Dcl-s Midnitial  Char(1)  Inz('B');
Dcl-s Lastname   Char(7)  Inz('Anthony');

Greeting = 'Hello ' + 'World';     // Greeting = 'Hello World '

Fullname = Firstname + ' ' + Midinitial + '. ' + Lastname;
                                   // Fullname = 'Susan B. Anthony'
```

Concatenating Character Values

- Character expression can span several lines without continuation character
- Simple concatenation retains leading/trailing blanks

```
Dcl-s Disclaimer Char(80);
Dcl-s Fullname   Char(25);
Dcl-s Firstname  Char(10) Inz('Susan');
Dcl-s Midnitial  Char(1)  Inz('B');
Dcl-s Lastname   Char(10) Inz('Anthony');

Disclaimer = 'This report is proprietary and confidential, for '
             + Companyname + ' internal use only.';

Fullname = Firstname + ' ' + Midinitial + '. ' + Lastname;
                                  // Fullname = 'Susan      B. Anthony    ‘
```

Using Built in Functions with Character Data

- %Trim (Trim Characters at Edges)
- %Subst (Get or Set Substring)
- %Scan (Scan String)
- %Replace (Replace Character String)
- %Scanrpl (Scan and Replace Characters)
- %Xlate (Translate)
- %Check (Check Characters), %Checkr (Check Reverse)

%Trim (Trim Characters at Edges)

- %Trim (Trim Characters) function removes all leading and trailing blanks
  - %Trim(variable-name{:characters})
  - Returns variable-length character value

```
Dcl-s Fullname   Char(30);
Dcl-s Firstname  Char(20)  Inz('Susan');
Dcl-s Midnitial  Char(1)  Inz('B');
Dcl-s Lastname   Char(20)  Inz('Anthony');

Fullname = Firstname + ' ' + Midinitial + '. ' + Lastname;
                                  // Fullname = 'Susan                B. Anthon'

Fullname = %Trim(Firstname) + ' ' + Midinitial + '. ' + %Trim(Lastname);
                                  // Fullname = 'Susan B. Anthony              '
```

%Trim (Trim Characters at Edges)

- Three variations
  - %Trim removes both leading and trailing blanks
  - %Triml removes leading blanks only
  - %Trimr removes trailing blanks only

```
Dcl-s Fullname   Char(30);
Dcl-s Firstname  Char(20)  Inz(‘   Susan   ');
Dcl-s Midnitial  Char(1)  Inz('B');
Dcl-s Lastname   Char(20)  Inz(‘   Anthony   ');

Fullname = Firstname + ' ' + Midinitial + '. ' + Lastname;
                                  // Fullname = ‘   Susan             B.    Ant'

Fullname = %Trim(Firstname) + ' ' + Midinitial + '. ' + %Trim(Lastname);
                                  // Fullname = 'Susan B. Anthony              '
```

%Trim (Trim Characters at Edges)

- %Trim functions can trim characters other than blanks

```
Dcl-s Checkword Char(25) Inz('$*****7,654.32***');
Dcl-s Result    Char(20);

Result = Checkword;                 // Result = '$*****7,654.32***   '

Result = %Trim(Checkword:'*');      // Result = '$*****7,654.32***   '
                                    // Checkword contains leading $ and
                                    // trailing blanks, but no edge asterisks.

Result = %Trim(Checkword:'$*');     // Result = '7,654.32***         '
                                    // Checkword contains trailing blanks,
                                    // so trailing asterisks are not trimmed.

Result = %Trim(Checkword:'$* ');    // Result = '7,654.32            '
                                    // Edge $, * and blanks are all trimmed.
```

Using Variable Length Character Data

- VARCHAR data has declared (maximum) length and current length (dependent upon value)
  - Current length can change while program is running
- System allocates memory for declared length plus two or four bytes to store current length (four bytes are added for very long variables, longer than 65,535 bytes)
- Some character string processing is more efficient with VARCHAR data
  - Use when length is uncertain

```
Dcl-s Firstvar Varchar(10);  // system will allocate 12 bytes – 10 for the data, plus
                             // 2 to hold the actual current length of the data
Dcl-s Longvar  Varchar(16773100);  // would allocate 4 extra bytes to hold current length

Dcl-s Nextvar  Varchar(10:4);
```

Using Variable Length Character Data

- Variable length character data may not need %Trim function

```
Dcl-s Fullname   Char(30);
Dcl-s Firstname  Char(20)  Inz('Susan');
Dcl-s Midnitial  Char(10)  Inz('B');
Dcl-s Lastname   Char(20)  Inz('Anthony');

Fullname = %Trim(Firstname) + ' ' + %Trim(Midinitial) + '. ' + %Trim(Lastname);
                                  // Fullname = 'Susan B. Anthony              '
```

```
Dcl-s Fullname   Char(30);
Dcl-s Firstname  Varchar(20)  Inz('Susan');
Dcl-s Midnitial  Varchar(10)  Inz('B');
Dcl-s Lastname   Varchar(20)  Inz('Anthony');

Fullname = Firstname + ' ' + Midinitial + '. ' + Lastname;
                                  // Fullname = 'Susan B. Anthony              '
```

%Subst (Get or Set Substring)

- %Subst (Get or Set Substring) function extracts a portion of a character string
  - %Subst(string:start{:length})

```
Dcl-s Phone     Char(10) Inz('9705551212');
Dcl-s Areacode  Char(3);
Dcl-s Exchange  Char(3);
Dcl-s Extension Char(4);
Dcl-s Longexten Char(7);

Areacode = %Subst(Phone:1:3);     // Areacode = '970'
Exchange = %Subst(Phone:4:3);     // Exchange = '555'
Extension = %Subst(Phone:7);      // Extension = '1212'
Longexten = %Subst(Phone:7);      // Longexten = '1212   '
```

%Subst (Get or Set Substring)

- Can also use %Subst as result of assignment expression to change designated substring's value

```
Dcl-s Areacode Char(3)  Inz('406');
Dcl-s Phone    Char(10) Inz('9705551212');
Dcl-s Phone2   Char(10) Inz('9516547531');

%Subst(Phone:1:3) = '613';                  // Phone = '6135551212'
%Subst(Phone:1:3) = Areacode;               // Phone = '4065551212'
%Subst(Phone:4:3) = %Subst(Phone2:4:3);     // Phone = '4066541212'
```

%Scan (Scan String)

- %Scan (Scan String) looks for character or string of characters within a character string
  - %Scan(search-arg:string{:start})
- Returns unsigned integer value that represents next position of the search argument in source string
  - If search argument is not found, returns zero
  - Scans left to right
  - Scan is case sensitive
- Useful to inspect text data
  - Index text based on keywords

%Scan (Scan String)

```
Dcl-s Position Uns(5);

Dcl-s Fielda Char(3)  Inz('XY ');
Dcl-s Fieldb Char(10) Inz('QRSTUVWXYZ');
Dcl-s Fieldc Char(10) Inz('XY TUVWXY ');

Dcl-s Fullname Char(20) Inz('Doe/John');

Position = %Scan(Fielda:Fieldb);            // Position = 0, because 'XY ' (with a
                                            // trailing blank) is not in Fieldb
Position = %Scan(%Trim(Fielda):Fieldb);     // Position = 8

Position = %Scan(Fielda:Fieldc);            // Position = 1
Position = %Scan(Fielda:Fieldc:2);          // Position = 8

Position = %Scan('/':Fullname);             // Position = 4
Position = %Scan('John':Fullname);          // Position = 5
Position = %Scan('JOHN':Fullname);          // Position = 0
```

%Replace (Replace Character String)

- Replace (Replace Character String) changes character string value by replacing existing characters with new values
  - %Replace(new-string:old-string{:start{:length}})

```
Dcl-s First   Char(5)  Inz('John');
Dcl-s Initial Char(1)  Inz('J');
Dcl-s Middle  Char(7)  Inz(' Quincy');
Dcl-s Name    Char(17) Inz('B. Adams');

Name = %Replace(Initial:Name);        // Name = 'J. Adams          '
Name = %Replace(First:Name:1:2);      // Name = 'John Adams        '
Name = %Replace(Middle:Name:5:0);     // Name = 'John Quincy Adams'
```

%Scanrpl (Scan and Replace Characters)

- %Scanrpl (Scan and Replace Characters) replaces every existing occurrence of value with new value
  - %Scanrpl(old-value:new-value:string{:start{:length})
  - Result is new character string with substituted values

```
Dcl-s String1 Char(48) Inz('SUBJECT sat on a wall. SUBJECT had a great fall.');
Dcl-s String2 Char(64);
Dcl-s Egg     Char(13) Inz('Humpty Dumpty');

String2 = %Scanrpl(Egg:'SUBJECT':String1);
  // String 2 =
  // 'Humpty Dumpty sat on a wall. Humpty Dumpty had a great fall.    '
```

%Xlate (Translate)

- %Xlate (Translate) converts characters string to other characters
  - %Xlate(from:to:string{:start})
  - Converts characters in *from* string to corresponding characters in *to* string
    - *From* and *to* strings must have same number of characters
  - A common use of this is to convert upper to lower or vice versa

```
Dcl-c Lowercase 'abcdefghijklmnopqrstuvwxyz';
Dcl-c Uppercase 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';

Dcl-s Fielda   Char(9)  Inz('abc123def');
Dcl-s Lastname Char(15) Inz('BYRNE-SMITH');
Dcl-s Name     Char(15);

Name = %Xlate(Uppercase:Lowercase:Lastname);      // Name = 'byrne-smith    '
Name = %Xlate(Uppercase:Lowercase:Lastname:2);    // Name = 'Byrne-smith    '
Name = %Xlate(' ':'*':Name);                      // Name = 'Byrne-smith****'
Fielda = %Xlate(Lowercase:Uppercase:Fielda);      // Fielda = 'ABC123DEF'
```

%Xlate (Translate)

```
Dcl-s Amount    Packed(11:2);
Dcl-s Checkword Char(17) Inz('$*****7,654.32***');

Checkword = %Xlate('$*,' : '   ' : Checkword); // Checkword = '      7 654.32   '
Amount = %Dec(Checkword:11:2);                 // Amount = 7654.32
```

%Check (Check Characters)

- %Check (Check Characters) inspects character string, checking whether it contains *only* characters designated in compare string
  - %Check(comparator:string{:start})
- Returns zero when all characters in string also appear in compare string
  - Otherwise, returns position of leftmost mismatch

```
Dcl-c Numbers ' .0123456789';

Dcl-s Checkword Char(25) Inz('      7,654.32');
Dcl-s Position  Uns(5);

Position = %Check(Numbers:Checkword);      // Position = 8
%Subst(Checkword:Position:1) = *Blank;     // Put a blank in position 8
Position = %Check(Numbers:Checkword);      // Position = 0
```

%Check (Check Reverse)

- %Checkr (Check Reverse) inspects from right to left
  - Returns position of rightmost mismatch

```
Dcl-s Name   Char25) Inz('Jones');
Dcl-s Length Uns(5);

Length = %Checkr(' ':Name);     // Length = 5
```

Data Conversion Functions

- %Char (Convert to Character Data)
- %Editc (Edit with an Edit Code)
- %Editw (Edit with an Edit Word)

%Char (Convert to Character Data)

- %Char (Convert to Character Data) converts numeric value to character value
  - Removes leading zeros
  - Return value includes leading sign for negative value

```
Dcl-s Message Char(45);
Dcl-s Points  Uns(10)     Inz(50273);
Dcl-s Balance Packed(9:2) Inz(-9876.54);

Message = 'You have earned ' + %Char(Points) + ' Frequent Flier Miles.';
             // Message = 'You have earned 50273 Frequent Flier Miles.  '

Message = 'Your account balance is ' + %Char(Balance);
             // Message = 'Your account balance is -9876.54             '
```

%Editc (Edit with an Edit Code)

- %Editc (Edit with an Edit Code) converts number to a character value, assigns edit code
  - Optionally adds currency symbol or asterisk leading fill (*****)

```
Dcl-s Message     Char(45);
Dcl-s Balance     Packed(9:2) Inz(9876.54);
Dcl-s Zerobalance Packed(9:2);

Message = 'Your account balance is ' + %Editc(Balance:'J':*Cursym);
                  // Message = 'Your account balance is   $9,876.54          '
Message = 'Your account balance is ' + %Trim( %Editc(Balance:'J':*Cursym) );
                  // Message = 'Your account balance is $9,876.54            '

Message = 'Your account balance is ' + %Editc(Zerobalance:'J':*Cursym);
                  // Message = 'Your account balance is        $.00          '
Message = 'Your account balance is ' + %Trim( %Editc(Zerobalance:'J':*Cursym) );
                  // Message = 'Your account balance is $.00                 '
```

%Editc (Edit with an Edit Code)

- To retain leading zeros, use X edit code
  - Does not reveal decimal point
  - Does not reveal explicit sign
    - Sign is embedded in rightmost digt

```
Dcl-s Var1    Char(15);
Dcl-s Number1 Packed(9:0) Inz(98765);
Dcl-s Number2 Packed(9:2) Inz(-98765.42);

Var1 = %Char(Number1);          // Var1 = '98765          '
Var1 = %Editc(Number1:'X');     // Var1 = '000098765      '

Var1 = %Char(Number2);          // Var1 = '-98765.42      '
Var1 = %Editc(Number2:'X');     // Var1 = '00987654K      '
```

%Editw (Edit with an Edit Word)

- %Editw (Edit with an edit word) applies edit word to a numeric value, converts number to character string

```
Dcl-c Edita '   ,   ,  **DOLLARS*  *CENTS';
Dcl-c Editp '0(   )   -    ';
Dcl-c Editt '   -  -    ';

Dcl-s Var1 Char(30);

Dcl-s Amount Packed(11:2) Inz(12345.67);
Dcl-s Phone  Packed(11:0) Inz(8005551212);
Dcl-s Taxid  Packed(9:0)  Inz(987654321);

Var1 = %Editw(Amount:Edita);           // Var1 = '*****12,345*DOLLARS*67*CENTS  '
Var1 = %Editw(Phone:Editp);            // Var1 = ' (800)555-1212                '
Var1 = %Editw(Taxid:Editt);            // Var1 = '987-65-4321                   '
Var1 = %Editw(Taxid:'   &  &    ');    // Var1 = '987 65 4321                   '
```

Retrieving Data Properties

- %Size (Byte Size)
- %Len (Get/Set Length)
- %Decpos (Number of Decimal Positions)

%Size (Byte Size)

- %Size (Byte Size) returns number of bytes occupied by variable or constant
  - Declared byte size
  - Varchar variables return declared size plus two or four bytes

```
Dcl-s Bytes   Uns(5);
Dcl-s Company Zoned(9:0)  Inz(68600);
Dcl-s Salary  Packed(9:0) Inz(68600);
Dcl-s Name    Char(25)    Inz('John Doe');
Dcl-s Namevar Varchar(25) Inz('Al Fredo');

Bytes = %Size(Bytes);       // Bytes = 2  (integer)
Bytes = %Size(Company);     // Bytes = 9  (zoned decimal)
Bytes = %Size(Salary);      // Bytes = 5  (packed decimal)
Bytes = %Size(Name);        // Bytes = 25
Bytes = %Size(Namevar);     // Bytes = 27 (declared size + 2)
```

%Len (Get/Set Length)

- %Len (Get/Set Length) returns number of characters in character variable, constant, or expression
  - Varchar variables return current length (dependent upon value)

```
Dcl-s Length    Int(5);

Dcl-s Name      Char(25)    Inz('John Doe');
Dcl-s Namevar   Varchar(25) Inz('Al Fredo');
Dcl-s Firstname VarChar(10) Inz('Frieda');
Dcl-s Lastname  Varchar(10) Inz('Slaves');

Length = %Len(Name);                // Length = 25
Length = %Len( %Trim(Name) );       // Length = 8

Length = %Size(Namevar);            // Length = 27 (declared size +2)
Length = %Len(Namevar);             // Length = 8  (current length)
Length = %Len(Namevar:*Max);        // Length = 25 (maximum length)
```

%Len (Get/Set Length)

- %Len (Get/Set Length) returns number of digits in numeric variable, constant, or expression

```
Dcl-s Length  Uns(5);

Dcl-s Company Zoned(5:0);
Dcl-s Salary  Packed(9:2) Inz(68600.00);
Dcl-s Tenure  Packed(3:0);

Length = %Len(Length);              // Length = 5  (integer)
Length = %Len(Company);             // Length = 5  (zoned decimal)
Length = %Len(Salary);              // Length = 9  (packed decimal)
Length = %Len(Salary * Tenure);     // Length = 12
```

%Len (Get/Set Length)

- Can use %Len to set current length (and value) of Varchar variable

```
Dcl-s Length Uns(5);
Dcl-s Name   Varchar(25) Inz('John Doe');

Length = %Len(Name);     // Length = 8 (current length)
%Len(Name) = 4;          // Name = 'John'
```

%Decpos (Number of Decimal Positions)

- %Decpos (Number of Decimal Positions) returns number of decimal positions in numeric variable or expression

```
Dcl-s Decimals Uns(5);
Dcl-s Digits   Uns(5);
Dcl-s Salary   Packed(9:2) Inz(68600);

Digits = %Len(Salary);          // Digits = 9
Decimals = %Decpos(Salary);     // Decimals = 2
```

Legacy Section – Glance through for now
Legacy Code: Move

- Move operation assigns Factor 2 value to Result variable
  - Fixed format only
  - Moves characters from right to left
- If Result is longer than Factor 2, extra leftmost Result characters are unaffected
  - If Result is shorter, contains only rightmost characters

Legacy Code: Move

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
DName++++++++++++ETDsFrom+++To/Len+IDc.Keywords+++++++++++++++++++++++++++++++
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

D Source          S              10    Inz('ABCDE12345')
D Equal           S              10    Inz('67890vwxyz')
D Short           S               5    Inz('FgHiJ')
D Long            S              15    Inz('kLmNoPqRsTu9876')

C                   Move      Source        Equal
C                   Move      Source        Short
C                   Move      Source        Long
                  // After Moves: Equal = 'ABCDE12345'
                  //               Short = '12345'
                  //               Long =  'kLmNoABCDE12345'
```

Legacy Code: Movel (Move Left)

- Movel (Move Left) operation works like Move, except it assigns values from left to right

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
DName++++++++++++ETDsFrom+++To/Len+IDc.Keywords+++++++++++++++++++++++++++++
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

D Source          S              10    Inz('ABCDE12345')
D Equal           S              10    Inz('67890vwxyz')
D Short           S               5    Inz('FgHiJ')
D Long            S              15    Inz('kLmNoPqRsTu9876')

C                   Movel     Source         Equal
C                   Movel     Source         Short
C                   Movel     Source         Long
                  // After Moves: Equal = 'ABCDE12345'
                  //               Short = ‘67890'
                  //               Long =  'ABCDE12345u9876'
```

Legacy Code: Padding Move Results

- (P) extender pads extra characters in the target with blanks
  - Movel(p) is similar to Eval operation
  - Move(p) is similar to Evalr operation

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
DName++++++++++ETDsFrom+++To/Len+IDc.Keywords+++++++++++++++++++++++++++++++
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

D Source          S              10    Inz('ABCDE12345')
D Long            S              15    Inz('kLmNoPqRsTu9876')

C                   Movel(p)  Source        Long
                                              // Long = 'ABCDE12345     '
C                   Move(p)   Source        Long
                                              // Long = '     ABCDE12345'
```

Legacy Code: Using Move with Numeric Data

- If both Factor 2 and Result are numeric, Move and Movel ignore decimal alignment
  - Use Z-add instead

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
DName++++++++++++ETDsFrom+++To/Len+IDc.Keywords+++++++++++++++++++++++++++++++
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

D Source          S                5  2 Inz(123.45)
D Target          S                5  1

C                   Move      Source         Target
                                              // After Move: Target = 1234.5
C                   Z-add     Source         Target
                                              // After Z-add: Target = 123.4
```

Legacy Code: Using Move to Convert Data

- Move and Movel can assign values to other data types, implicitly converting Factor 2 value to Result data type
  - If Factor 2 is character and Result is numeric, each character's digit portion is converted to corresponding numeric character, then moved to Result
    - Blanks become zeros in the result
    - Rightmost character's zone portion is sign of Result

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
DName++++++++++++ETDsFrom+++To/Len+IDc.Keywords+++++++++++++++++++++++++++++
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

D Source          S                5    Inz(' 123N')
D Target          S                5  2

C                   Move      Source        Target
                                            // After Move: Target = -012.34
```

Legacy Code: Resulting Indicators

- Resulting indicators in positions 71–76 (HiLoEq) reflect result of move
  - If Result is numeric…
    - Hi indicator (71–72) is set *On for positive result
    - Lo indicator (73–74) is set *On for negative result
    - Eq indicator (75–76) is set *On for zero result
  - If Result is character…
    - Eq indicator (75-76) is set on if result is blank

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C                   Move      Source        Target                    50
```